

HTML Learning Portal

Unit 1 Study Guide

Introduction to Web Technologies & HTML Fundamentals

16 topics · condensed key concepts · exam-style practice questions with answers

How to use this guide

Part 1 is a condensed, topic-by-topic reference — read it as a quick refresher, not a first-time introduction (for that, use the full tutorials on the site). Part 2 is a set of exam-style practice questions covering the whole unit, in several formats you're likely to see on a real exam: short answer, differentiate-between, write-the-code, fill-in-the-blank, and true/false. Answers are collected at the end so you can self-check without seeing them while you attempt the questions.

by Dr. Puneet Thapar — Associate Professor, School of Computing and Artificial Intelligence, Lovely Professional University

Part 1 — Quick Reference

1. Introduction to Internet and Web Technologies

- The Internet is a global network of interconnected computers; the Web is a system of linked documents (pages) accessed over it via URLs.
- A browser is the client that requests pages; a server stores site files and responds to those requests.
- A URL has a scheme, domain, optional port, path, and optional query string. DNS maps a human-readable domain name to a numeric IP address.
- HTTP is the protocol for exchanging requests/responses; HTTPS is HTTP encrypted with TLS/SSL.
- Static sites serve fixed files identical for every visitor; dynamic sites generate pages on the server per request.

2. Client–Server Architecture

- The client requests a resource (usually a browser); the server processes the request and returns a response.
- A request has a method (GET, POST...), a URL, headers, and sometimes a body. A response has a status code (200, 404, 500...), headers, and a body.
- HTTP lifecycle: connect → send request → server processes → server sends response → connection closes/reused.
- DNS resolution (translating the domain to an IP) happens before the request can even be sent.

3. HTML5 Fundamentals

- HTML5 is the current standard version of HTML, adding semantic tags, native audio/video, canvas & SVG, and built-in form validation.
- HTML evolved through 2.0 → 4.01 → XHTML → HTML5, each version adding capability.
- HTML5 reduces reliance on plugins (like Flash), improves accessibility via semantics, and gives more consistent cross-browser behavior.

4. HTML Document Structure

- `<!DOCTYPE html>` declares HTML5 and triggers the browser's standards mode.
- `<html lang="...">` is the root element wrapping everything else.
- `<head>` holds non-visible metadata: `<title>`, `charset`, `stylesheets`, `scripts`, `meta` tags.
- `<body>` contains everything the visitor actually sees and interacts with.

5. HTML Elements

- Most elements have an opening and matching closing tag; void/empty elements (`<img>`, `<br>`, `<hr>`) never wrap content and have no closing tag.
- Nested elements must close in the exact reverse order they were opened.
- Block elements (`<div>`, `<p>`) start on a new line; inline elements (`<span>`, `<a>`) flow within a line of text.

6. HTML Attributes

- An attribute adds information to a tag: `name="value"` inside the opening tag.
- `id` is a unique identifier for exactly one element; `class` is a reusable label that can apply to many elements.
- `style` applies inline CSS to one element (fine for quick tests, avoid in production); `title` adds a hover tooltip and extra accessibility context.

7. Text Formatting

- `<h1>`–`<h6>` establish the page outline in decreasing importance; `<p>` is a standard paragraph block.
- `<strong>`/`<em>` carry semantic meaning (importance/emphasis); `<b>`/`<i>` are purely visual with no semantic weight.
- `<mark>` highlights, `<small>` de-emphasizes, `<sub>`/`<sup>` position text below/above the baseline.

- `<blockquote>` marks a quoted passage; `<pre>` preserves whitespace and line breaks exactly as written.

8. Lists

- `<ul>` = unordered (bulleted) list; `<ol>` = ordered (numbered, with `type/start` attributes); each item is an `<li>`.
- `<dl>` pairs a term (`<dt>`) with a definition (`<dd>`) — ideal for glossaries.
- Any list can be nested inside one of its own `<li>` items to form a sub-list.

9. Hyperlinks

- `<a href="...">` creates a clickable link; `href` names the destination.
- Internal links use relative paths; external links typically use a full `https://` URL.
- `mailto:` opens the visitor's email client; `target="_blank"` opens a new tab (pair with `rel="noopener"` for security).

10. Images

- `<img>` is a void element that always needs `src` and `alt`.
- `width/height` reserve layout space and prevent content from jumping as the image loads.
- `srcset/sizes` let the browser choose the best image file for the viewer's screen; `<figure>/<figcaption>` associate an image with a caption semantically.

11. SVG Graphics

- SVG (Scalable Vector Graphics) describes images mathematically, so they stay crisp at any zoom level.
- Core shape elements: `<circle>`, `<rect>`, `<line>`, `<polygon>`, `<text>`.
- `viewBox` defines the coordinate system shapes are drawn in, independent of the element's displayed size.

12. Audio

- `<audio>` embeds sound; controls shows native play/pause/volume UI.
- `autoplay` starts playback automatically (usually requires muted); `loop` repeats playback.
- Nesting multiple `<source>` elements lets the browser pick a format it supports.

13. Video

- `<video>` embeds a video file with native playback controls.
- `poster` sets a preview image shown before playback starts; `width/height` (or CSS aspect-ratio) reserve layout space.

14. Iframes

- `<iframe src="...">` nests another HTML document inside the current page — used for embedding external sites, videos, and maps.
- The `sandbox` attribute restricts what an embedded page can do (scripts, forms, popups, etc.), reducing risk from third-party content.

15. SEO Fundamentals

- SEO (Search Engine Optimization) structures content so search engines can understand and rank it well.
- `<title>` and the meta description are the two most visible elements in search results.
- Semantic HTML and accessibility practices (alt text, logical heading order, descriptive link text) both improve search visibility.

16. HTML Best Practices

- Prefer semantic elements (`nav`, `main`, `article`) over generic `div/span` for structural content.
- Provide alt text, label form fields, keep a logical heading order, and ensure sufficient color contrast.
- Separate structure (HTML), presentation (CSS), and behavior (JS) into their own files; validate markup to catch unclosed tags and invalid nesting.

Part 2 — Exam Practice Questions

Section A — Short Answer

- Q1. Define the Internet and the World Wide Web, and explain how they differ.
- Q2. What is the role of DNS in loading a web page?
- Q3. Explain the difference between a static and a dynamic website.
- Q4. What does HTTPS add on top of plain HTTP, and why does it matter?
- Q5. List three features that HTML5 introduced over earlier HTML versions.
- Q6. What is the purpose of the `<!DOCTYPE html>` declaration?
- Q7. What is a void (empty) element? Give two examples.
- Q8. Explain the difference between the `id` and `class` attributes.
- Q9. Why should width and height be set on an `<img>` even though CSS can resize it?
- Q10. What is the purpose of the `sandbox` attribute on an `<iframe>`?

Section B — Differentiate Between

- Q11. [Differentiate] `<strong>` vs. `<b>`, and `<em>` vs. `<i>`
- Q12. [Differentiate] Block-level elements vs. inline elements
- Q13. [Differentiate] `<ul>` vs. `<ol>` vs. `<dl>`
- Q14. [Differentiate] Internal links vs. external links
- Q15. [Differentiate] GET request vs. POST request (in terms of what they're used for)

Section C — Write the Code

- Q16. [Code] Write the minimal valid skeleton of an HTML5 document, including the language attribute and character encoding.
- Q17. [Code] Write an `<a>` tag that opens `https://example.com` in a new tab safely.
- Q18. [Code] Write an `<img>` tag for a photo at `photo.jpg` with meaningful alt text and explicit width/height.
- Q19. [Code] Write a nested unordered list: a top-level item containing a 2-item sub-list.
- Q20. [Code] Write an `<audio>` tag with native controls and two `<source>` elements for different formats.

Section D — True or False

- Q21. [T/F] `<br>` requires a matching closing tag.
- Q22. [T/F] A page should have exactly one `<title>` element inside `<head>`.
- Q23. [T/F] The `class` attribute must be unique to one element per page.
- Q24. [T/F] HTTPS uses TLS/SSL encryption to protect data between browser and server.

Q25. [T/F] srcset lets the browser pick which image file to load based on screen size.

Answer Key

- A1.** The Internet is the global network of interconnected computers; the Web is a system of linked documents/pages accessed over that network via URLs and HTTP. The Web is one of many services that run on the Internet (email and file transfer are others).
- A2.** DNS translates the human-readable domain name typed/clicked by the user into the numeric IP address of the server, which is required before the browser can actually send a request to it.
- A3.** A static site serves fixed files that look identical for every visitor. A dynamic site generates the page content on the server (often from a database) per request, so it can differ by user or context.
- A4.** HTTPS is HTTP encrypted with TLS/SSL, protecting data in transit from eavesdropping or tampering — important for login credentials, payments, and general user privacy/trust.
- A5.** Any three of: semantic tags (header/nav/article/etc.), native audio/video without plugins, canvas & SVG graphics, built-in form validation, offline storage APIs.
- A6.** It declares the document as HTML5 and puts the browser into standards mode, ensuring consistent, predictable rendering.
- A7.** A void element never wraps content and has no closing tag. Examples: `<img>`, `<br>`, `<hr>`, `<input>`.
- A8.** `id` must be unique to exactly one element on the page (used for anchors/labels/scripting a single element); `class` is reusable across many elements (used for styling/scripting a group).
- A9.** Setting `width/height` reserves the image's layout space before it finishes loading, preventing surrounding content from jumping around (a poor user experience and a page-quality signal).
- A10.** `sandbox` restricts what the embedded page can do — e.g. run scripts, submit forms, open popups — reducing the risk of third-party content affecting the parent page.
- B1.** `<strong>/<em>` carry semantic meaning (importance/emphasis) understood by screen readers and search engines; `<b>/<i>` are purely visual with no semantic weight, even though they may render identically.
- B2.** Block elements start on a new line and take the full available width by default (`<div>`, `<p>`); inline elements flow within a line of surrounding text and only take up as much width as their content (`<span>`, `<a>`).
- B3.** `<ul>` is an unordered/bulleted list, `<ol>` is an ordered/numbered list, and `<dl>` is a description list pairing terms (`<dt>`) with definitions (`<dd>`) — not a simple item list at all.
- B4.** Internal links point to another page/section on the same site, usually with a relative path; external links point to a different site, typically with a full `https://` URL.
- B5.** GET requests data and appends parameters to the URL (visible, bookmarkable, used for searches/navigation); POST sends data in the request body (used for anything sensitive, large, or that changes server state, like form submissions).
- C1.** `<!DOCTYPE html><html lang="en"><head><meta charset="UTF-8"><title>Page</title></head><body></body></html>`
- C2.** `<a href="https://example.com" target="_blank" rel="noopener">Visit</a>`
- C3.** ``
- C4.** `<ul><li>Item one</li><li>Sub-item A</li><li>Sub-item B</li></ul></li></ul>`
- C5.** `<audio controls><source src="song.mp3" type="audio/mpeg"><source src="song.ogg" type="audio/ogg"></audio>`
- D1.** False — `<br>` is a void element with no closing tag.
- D2.** True.
- D3.** False — `class` is meant to be reused across many elements; `id` is the one meant to be unique.
- D4.** True.

D5. True.